

„They [don't] see me rollin' ...“¹



Abb. 0.1: Reaktionsräder vom Hubble Weltraumteleskop³

Auswertung Versuch 213 - Kreisel

Physikalisches Praktikum PAP 2.1

des Studienganges Physik
an der Universität Heidelberg

von

Benjamin Kleijn

3. April 2026

Versuchstermin	25.02.2026
Gruppe, Kurs	19, nachmittags
Tutor:in	Rudolf Erdmann Mey

Inhaltsverzeichnis

1. Einleitung	3
1.1. Motivation	3
1.2. Ziele des Versuches	3
1.3. Geräte	3
1.4. Theoretische und technische Grundlagen	4
1.5. Symmetrischer Kreisel	4
1.6. Kräftefreier, symmetrischer Kreisel	5
1.7. Schwerer Kreisel	6
1.8. Zusammenhang der Trägheitsmomente	7
2. Messverfahren und Messprotokoll	8
3. Auswertung	10
3.1. Dämpfungskonstante	10
3.2. Präzession	10
3.3. Beobachtung der Nutation	12
3.4. Berechnung von I_x	12
3.5. Berechnung I_x die zweite	13
4. Zusammenfassung und Diskussion	14
4.1. Fitfehler	14
4.2. Dämpfung	14
4.3. Berechnung von I_x	14
4.4. Vergleich der Methoden zur I_x Bestimmung	14
4.5. Fazit	15
Literaturquellen	17
Abbildungsquellen	17
A. Formeln der statistischen Analyse	18
B. Code-Anhang	18

1. Einleitung

1.1. Motivation

Während man sich in PAP1 noch mit der Einführung der Größe *Trägheitsmoment* beschäftigt hat, wird in diesem Versuch das Objekt KREISEL mit den Eigenschaften *Nutation* und *Präzession* genauer untersucht, sowie durch geeignete Messungen die relevanten Trägheitsmomente des Kreisel bestimmt. Ziel des Versuches ist es, Verständnis für Kreiseltheorie, Rotationsbewegungen und ihre physikalische Beschreibung zu vertiefen.

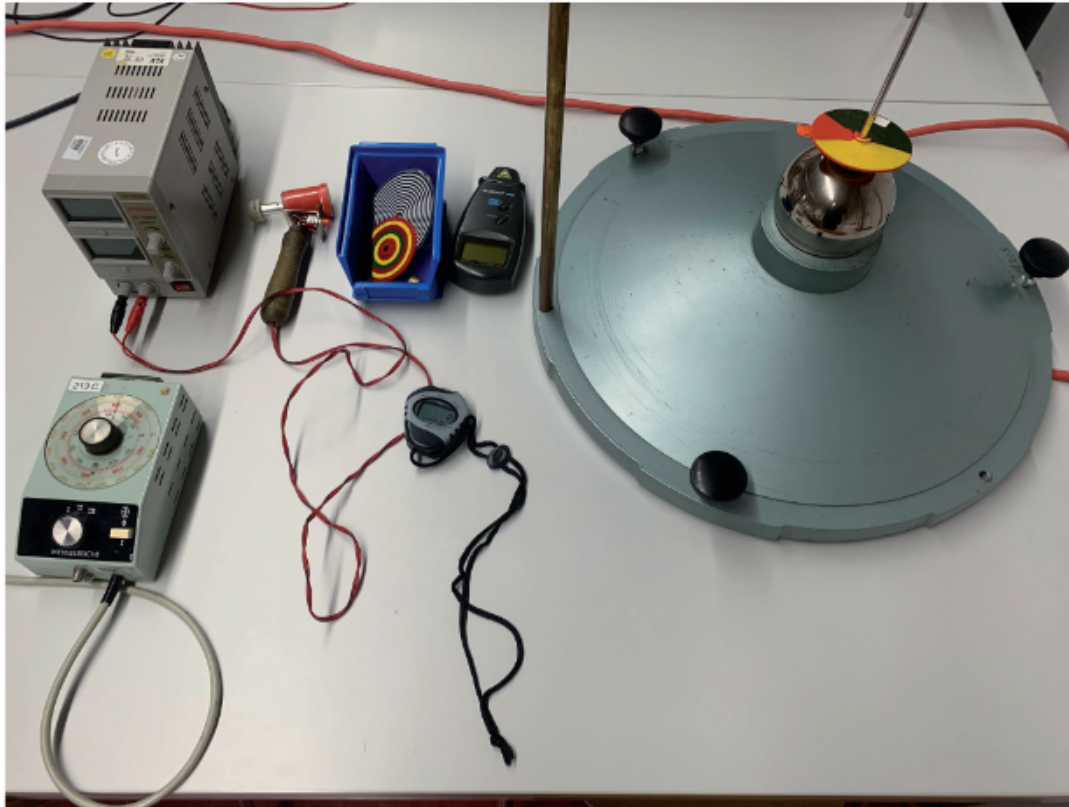
Anmerkung zum Meme: Reaktionsräder werden in der Raumfahrttechnik als elektrisch betriebenes Steuerungssystem eingesetzt, um keine mit Treibstoff laufenden Steuerungsdüsen zu benötigen. Für den Satellit gilt Drehimpulserhaltung, das heißt eine geänderte Drehzahl eines Reaktionsrades sorgt für ein Drehmoment, das den Satellit in die entgegengesetzte Richtung dreht, siehe Newtons 3. Gesetz. Dafür sind sie meist mit mehreren (auch redundanten) Rädern ausgestattet, die in verschiedenen Achsen gelagert sind, sodass jede Drehbewegung möglich ist.

1.2. Ziele des Versuches

- Verhalten des Kreisel qualitativ beobachten und damit vertraut werden
- Dämpfungskonstante und Halbwertszeit der Reibung bestimmen
- aus Präzessionsfrequenz eines schweren Kreisel das Trägheitsmoment I_z entlang der Figurenachse und das senkrecht dazu liegende Hauptträgheitsmoment $I_x = I_y$ bestimmen
- obiges Trägheitsmoment auch aus Nutation bestimmen

1.3. Geräte

1. Stahlkugel mit Aluminiumstab ($m = 4.164 \text{ kg}$ incl. Stab, Kugelradius $r = 5.08 \text{ cm}$) als KREISEL gelagert in einer Luftkissenpfanne
2. 2 Gewichte ($r_a = 0.725 \text{ cm}$, $r_b = 0.325 \text{ cm}$, $h = 1.1 \text{ cm}$, $m = 9.85 \text{ g}$)
3. Farbscheibe mit unterschiedlichen Farbbereichen
4. Stroboskop
5. Stoppuhr
6. Motor mit Netzgerät

Abb. 1.1: Versuchsaufbau/Geräte²

1.4. Theoretische und technische Grundlagen²

1.5. Symmetrischer Kreisel

Ein symmetrischer Kreisel ist ein starrer Körper, dessen zwei Hauptträgheitsmomente übereinstimmen, also

$$I_x = I_y \neq I_z$$

Die Symmetrieachse wird als *Figurenachse* bezeichnet. Zur Beschreibung der Bewegung werden drei Achsen unterschieden:

- die Figurenachse F ,
- die Drehimpulsachse L ,
- die momentane Drehachse $\vec{\omega}$.

Fallen diese drei Achsen zusammen, führt der Kreisel eine reine Eigenrotation mit Winkelgeschwindigkeit ω_F aus, siehe links in Abb. 1.2. Sobald jedoch ein Stoß oder eine äußere Störung erfolgt, treten Nutation und Präzession auf, wodurch die Achsen nicht länger parallel sind.

1.6. Kräftefreier, symmetrischer Kreisel

Ein Kreisel ist *kräftefrei*, wenn sein Auflagepunkt im Schwerpunkt liegt. Dann wirkt kein äußeres Drehmoment, sodass der Drehimpuls in Richtung und Betrag zeitlich konstant bleibt:

$$\frac{d\vec{L}}{dt} = 0$$

Wird ein rotierender Kreisel leicht angestoßen, beginnt die Figurenachse auf einem sogenannten *Nutationskegel* um die Drehimpulsachse zu rotieren, siehe rechte Seite in Abb. 1.2. Die momentane Winkelgeschwindigkeit ω setzt sich aus Eigenrotation ω_F und Nutation ω_N zusammen:

$$\vec{\omega} = \vec{\omega}_N + \vec{\omega}_F \quad (1.1)$$

Für kleine Nutationswinkel kann die Nutationsfrequenz näherungsweise durch

$$\omega_N \approx \frac{I_z}{I_x} \omega_F \quad (1.2)$$

beschrieben werden.

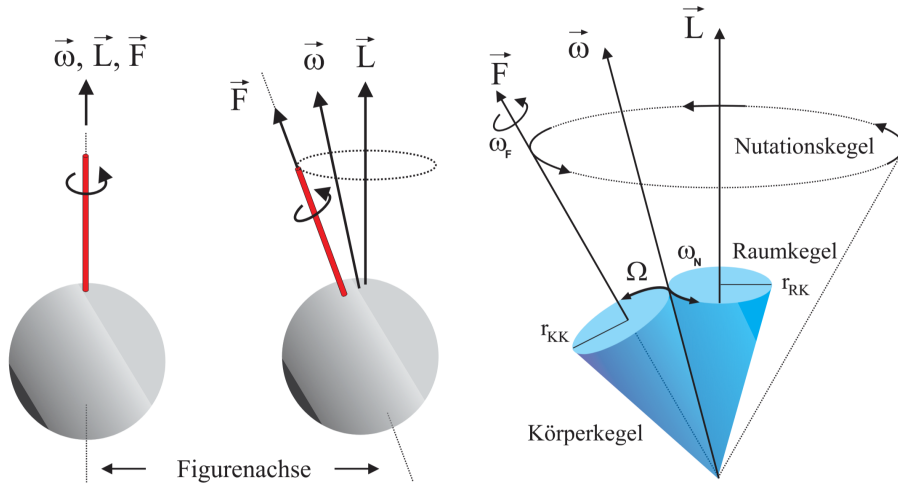


Abb. 1.2: Links: kräftefreier, symmetrischer Kreisel.
Rechts: Nutationsbewegung selbigen Kraisels nach einem Stoß: Achsen nicht mehr identisch und Körperkegel rollt auf Raumkegel ab²

Die momentane Drehachse wandert mit einer Winkelgeschwindigkeit Ω um die Figurenachse. Zwischen Ω , ω_N und ω_F gilt

$$\Omega = \omega_N \frac{r_{KK}}{r_{RK}} \quad (1.3)$$

wobei r_{KK} und r_{RK} die Radien des Körper- bzw. Raumkegels sind.

Zur Visualisierung der momentanen Drehachse und ihrer Rotation mit Ω auf dem Körperkegel, dient eine farblich segmentierte Scheibe, die zentrisch an der Figurenachse befestigt ist. Immer wenn die Figurenachse gerade in einem Farbsegment liegt, kann man diese Farbe erkennen, ansonsten vermischen sich alle Farben aufgrund der begrenzten Aufnahme Frequenz des Auges. Der Farbwechsel erfolgt mit der Frequenz Ω . Durch kompliziertes Umformen ergibt sich schließlich

$$\Omega = \left(1 - \frac{I_z}{I_x}\right) \omega_F \quad (1.4)$$

Diese Beziehung erlaubt später die Bestimmung von I_x aus der beobachteten Umlauffrequenz Ω .

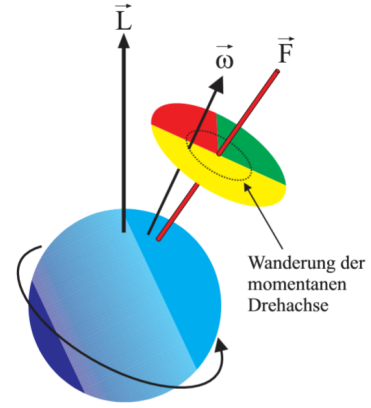


Abb. 1.3: Farbscheibe zur Bestimmung von Ω ²

1.7. Schwerer Kreisel

Wird der Kreisel nicht im Schwerpunkt gelagert, sondern durch ein Zusatzgewicht m entlang der Figurenachse belastet, entsteht ein äußeres Drehmoment

$$\vec{M} = \vec{l} \times m\vec{g} = mgl \sin(\alpha) \quad (1.5)$$

wobei l der Abstand der Zusatzmasse vom Auflagepunkt ist, g die Erdbeschleunigung und α der Winkel, in dem die Figurenachse ausgerichtet ist.

Dieses Drehmoment führt zu einer zeitlichen Änderung der Drehimpulsrichtung, wodurch der Kreisel eine *Präzessionsbewegung* ausführt.

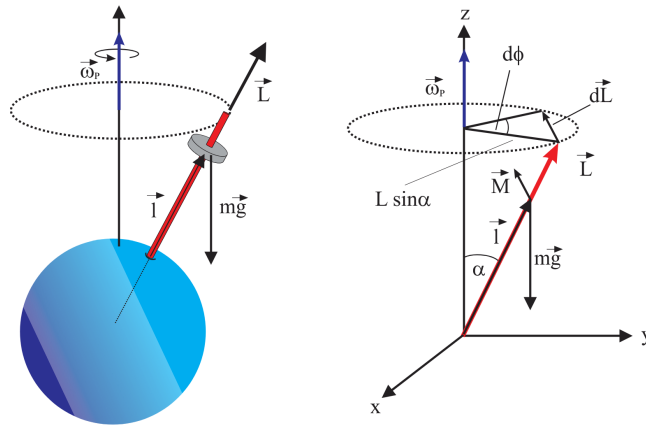


Abb. 1.4: Darstellung und Geometrie des schweren symmetrischen Kreisels zur Bestimmung der Präzessionsfrequenz ω_p ²

Die Präzessionsfrequenz ergibt sich zu

$$\omega_p = \frac{d\varphi}{dt} = \frac{dL}{L \sin(\alpha) dt} = \frac{mgl}{I_z \omega_F}. \quad (1.6)$$

Damit ist die Präzession umso schneller, je größer das erzeugte Drehmoment mgl ist, je kleiner das Trägheitsmoment I_z ist, je langsamer die Eigenrotation ω_F um die Figurenachse ist.

Treten Nutation und Präzession gleichzeitig auf, bewegt sich die Figurenachse auf einer girlandenförmigen Bahn (Nutation ist das Nicken des Kreisels im Altitudwinkel, Präzession das Rotieren im Azimuthwinkel). Für die quantitative Auswertung wird jedoch jeweils ein dominierender Effekt betrachtet.

1.8. Zusammenhang der Trägheitsmomente

Kennt man genügend Messgrößen, so lassen sich aus diesen die Trägheitsmomente bestimmen. Für die im Versuch definierten Achsen ergibt sich nach Gleichung (1.4) unter Kenntnis der Figurenfrequenz ω_F , der Winkelgeschwindigkeit Ω :

$$I_x = I_y = \frac{\frac{\omega_F}{\Omega} I_z}{\frac{\omega_F}{\Omega} - 1} = \frac{m I_z}{m - 1} \quad (1.7)$$

2. Messverfahren und Messprotokoll

Dämpfung Für alle Versuchsteile soll die Kugel zusammen mit dem angebrachten Farbschirm kräftefrei sein. Dafür wird die im Luftkissen gelagerte Kugel auf 90° gekippt, im kräftefreien Fall sollte dann bei geringer Eigenrotation keine Präzessions auftreten, ansonsten wird die Justierungshöhe der Farbscheibe verschoben.

Zur Bestimmung der Dämpfungsstärke über zwölf Minuten die Rotationsgeschwindigkeit gemessen. Dabei wird die Kugel zunächst mit dem Motor auf ca. 700 rpm beschleunigt und dann jede Minute mit dem Stroboskop die Frequenz gemessen.

Tabelle 2.1: Reibungsmessung

t [s]	0	60	120	180	240	300	360	$\Delta t = 2$ s
f_F [rpm]	650	596	543	499	458	422	386	$\Delta f = 5$ rpm

Präzession An den Kreisel mit um 45° ausgelenkter Stange werden verschiedene Gewichte (vier Gewichtskonfigurationen) befestigt. Die sich einstellende Präzessionsfrequenz wird durch Messen der Periodendauer mit der Stoppuhr bestimmt. Dies erfolgt für drei verschiedene Frequenzen ω_F . Der Auslenkwinkel ist nicht relevant, da die Präzession unabhängig von diesem ist.

Tabelle 2.2: Präzessionsfrequenz

Masse m -Abstand d $\Delta l = 3$ mm	Winkel $[\circ]$	T_p [s] $\Delta T_p = 0.5$ s	f_F [rpm] $\Delta f_F = 5$ rpm
$m - 20$ cm	80	76.3	486
	45	75.8	493
	45	107.2	694
	45	49.9	313
$m - 15$ cm	45	142.9	682
	45	99.7	490
	45	56.6	269
$2m - 20$ cm	45	50.2	670
	45	36.6	490
	45	23.3	315
$2m - 15$ cm	45	64.4	630
	45	49.2	495
	45	27.9	280

Messung der Nutation Zur Beobachtung der Nutation wurde der Kreisel kräftefrei betrieben und durch Ein leichten Stoß am Stabanfang an den kräftefreien Kreisel initiiert die Nutationsbewegung. Für fünf verschiedene Eigenfrequenzen wurde die Zeit für zehn vollständige Farbwechsel gemessen, woraus sich die Umlauffrequenz Ω ergibt.

Tabelle 2.3: Nutation - Ω Messung

f_F [rpm] $\Delta f_F = 5$	f_F [Hz] $\Delta f_F = 0.08$	$10T_\Omega$ [s] $\Delta T = 0.5$
570	9.50	18.2
480	8.00	22.1
362	6.03	29.2
292	4.87	35.4
400	6.67	26.7

Zusätzlich wurde die Nutationsfrequenz direkt mit dem Stroboskop bestimmt. Dazu wurde das Stroboskop zuerst so eingestellt, dass die durch Nutation rotierende Stange scheinbar stillsteht, und unmittelbar danach leicht reduziert, um die Eigenfrequenz abgelesen. Da der Kreisel gedämpft ist, mussten beide Messungen schnell hintereinander erfolgen.

Tabelle 2.4: Nutation - Stroboskopmessung

f_F [rpm]	870	740	710	680	640	630	617	517	430	330	285	$\Delta f_F = 10$
f_F [rpm]	815	700	670	640	605	600	582	490	400	305	265	$\Delta f_F = 10$

3. Auswertung

3.1. Dämpfungskonstante

Die Werte aus Tabelle 2.1 werden von [rpm] in $[\frac{\text{rad}}{\text{s}}]$ umgerechnet und dann in einem logarithmischen Graphen aufgetragen. Da erwartet wird, dass exponentielle Abnahme vorliegt:

$$\omega_F(t) = b \cdot e^{-\delta t} \quad \ln(\omega_F) = \ln(b) - \delta t \quad (3.1)$$

lässt sich eine Gerade $\ln(y) = -\delta t + \ln(b)$ durch die logarithmischen aufgetragenen Messwerte fitten, deren Parameter δ die Dämpfungskonstante ist, $\Delta\delta$ wird aus dem Fitfehler extrahiert. Zu beachten ist jedoch, dass der Fehler $\Delta^{\ln}\omega_F = \frac{\partial \ln(\omega_F)}{\partial \omega_F} \Delta\omega_F = \frac{\Delta\omega_F}{\omega_F}$ beim Fitten wie hier dargestellt umgewandelt werden muss, gemäß Gauß'scher Fehlerfortpflanzung A.5. Mithilfe der Dämpfungskonstante kann noch die Halbwertszeit $T_{1/2}$ berechnet werden:

$$T_{1/2} = \frac{\ln(2)}{\delta} \quad \Delta T_{1/2} = \frac{\ln(2)}{\delta^2} \Delta\delta \quad (3.2)$$

$$\delta = (145 \pm 4) \cdot 10^{-5} \frac{1}{\text{s}} \quad T_{1/2} = (480 \pm 13) \text{s}$$

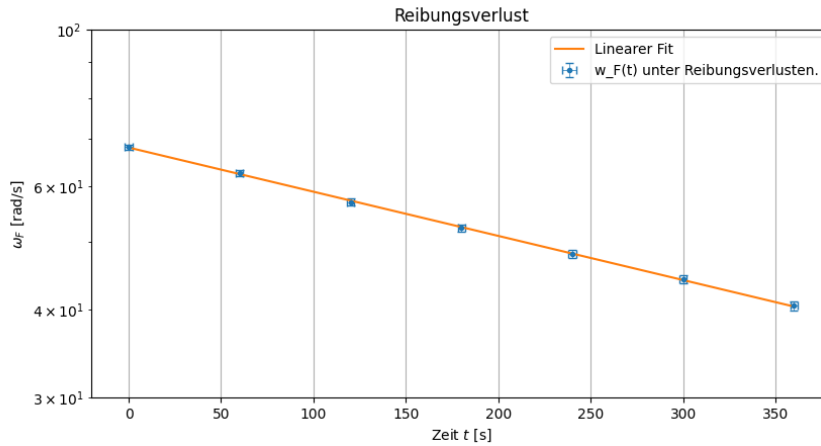


Abb. 3.1: Ermittlung der Reibungskonstante durch logarithmisches Auftragen

3.2. Präzession

Man sieht an der ersten Zeile in Tabelle 2.2, dass die Präzessionsdauer unabhängig des Winkels der Figurenachse ist, die Abweichung der beiden Werte beträgt nur 1.2σ .

Zur Bestimmung des Trägheitsmoments I_z wird Gleichung (1.6) verwendet. ω_P wird aus den Periodendauern erhoben. Die Frequenz ω_F ändert sich während der Präzessionsdauer, deswegen wird die Anfangsfrequenz ω_F^i mittels der Dämpfungskonstante in eine Endfrequenz

ω_F^f umgerechnet, und dann der Mittelwert $\bar{\omega}_F$ gebildet. Natürlich werden hiervoor die mithilfe des Stroboskops gemessenen Frequenzen f_F^i in Kreisfrequenzen umgewandelt.

$$\omega_P = \frac{2\pi}{T_P} \quad \Delta\omega_P = \frac{2\pi}{T_P^2} \Delta T_P \quad (3.3a)$$

$$\omega_F^i = 2\pi f_F^i \quad \Delta\omega_F^i = 2\pi \Delta f_F^i \quad (3.3b)$$

$$\omega_F^f = \omega_F^i \cdot e^{-\delta T_P} \quad \Delta\omega_F^f = e^{-\delta T_P} \sqrt{(\Delta\omega_F^i)^2 + (\omega_F^i \delta \Delta T_P)^2 + (\omega_F^i T_P \Delta \delta)^2} \quad (3.3c)$$

$$\bar{\omega}_F = \frac{\omega_F^i + \omega_F^f}{2} \quad \Delta\bar{\omega}_F = \frac{1}{2} \sqrt{(\Delta\omega_F^i)^2 + (\Delta\omega_F^f)^2} \quad (3.3d)$$

So wurden alle benötigten Werte berechnet. Jetzt werden pro äußerem Drehmoment (es gab 4 Konfigurationen) $T_P(\omega_F)$ in ein Diagramm aufgetragen und eine Gerade gefittet. Die Steigungen ergeben sich zu:

$$a_{m-20} = (1.602 \pm 0.020) \text{ s}^2$$

$$a_{m-15} = (2.275 \pm 0.019) \text{ s}^2$$

$$a_{2m-20} = (0.763 \pm 0.021) \text{ s}^2$$

$$a_{2m-15} = (1.061 \pm 0.021) \text{ s}^2$$

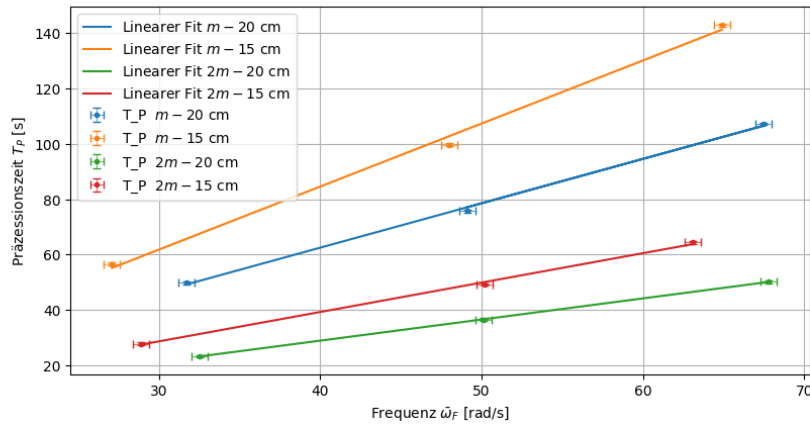


Abb. 3.2: Steigungsbestimmung der Präzessionsdauer $T_P(\omega_F)$

Diese Steigungen werden gemäß Gleichung (1.6) genutzt, um die Trägheitsmomente I_z zu bestimmen. Die Trägheitsmomente der präzessionserzeugenden Gewichtsstücke können vernachlässigt werden. Da die Kugel selber so gelagert ist, dass sie sich um ihren Schwerpunkt dreht, trägt sie nicht zum Drehmoment bei, welches die Präzession erzeugt.

$$T_P = \frac{\overbrace{2\pi I_z}^{=a}}{mgl} \bar{\omega}_F \quad \rightarrow \quad I_z = \frac{amgl}{2\pi} \quad \Delta I_z = I_z \sqrt{\frac{\Delta_a^2}{a^2} + \frac{\Delta_g^2}{g^2} + \frac{\Delta_l^2}{l^2}} \quad (3.4)$$

Mit den Längen aus Tabelle 2.2, der Heidelberger Erdbeschleunigung $g = (9.809\,84 \pm 0.000\,02) \frac{\text{m}}{\text{s}^2}$ und der Masse $m = 9.85 \text{ g}$ ergeben sich die folgenden Werte:

$$\begin{aligned} I_{m-20} &= (4.93 \pm 0.10) \cdot 10^{-3} \text{ kg m}^2 & I_{m-15} &= (5.25 \pm 0.12) \cdot 10^{-3} \text{ kg m}^2 \\ I_{2m-20} &= (4.70 \pm 0.15) \cdot 10^{-3} \text{ kg m}^2 & I_{2m-15} &= (4.90 \pm 0.14) \cdot 10^{-3} \text{ kg m}^2 \end{aligned}$$

Als Ergebnis wird der Mittelwert verwendet, dessen Fehler über quadratisches Addieren der Standardabweichung und des Mittelwerts der Einzelfehler bestimmt wurde.

$$I_z := \bar{I}_z = (4.95 \pm 0.14) \cdot 10^{-3} \text{ kg m}^2$$

3.3. Beobachtung der Nutation

Die Farbwechselrichtung (Ω) wurde zwar aufgeschrieben: „grün - rot - gelb“, jedoch ist die Umlaufrichtung der Figurenachse (ω_F) nicht mehr bekannt. Ist diese gleichgerichtet mit der Rotation der Farbscheibe, so ist $\frac{\Omega}{\omega_F} > 0$. Dies sorgt nach Gleichung (1.4) dafür, dass $1 - \frac{I_z}{I_x} > 0$, daraus folgt $I_x > I_z$, wie dies im Skript² steht.

Wenn Gleichung (1.7) betrachtet wird, um eine Aussage zu der im nächsten Abschnitt betrachteten Steigung des Fits zu machen, so entsteht folgendes: We denote $\frac{m}{m-1} := V$:

$$\text{für } m = \frac{\omega_F}{\Omega} \quad \left\{ \begin{array}{ll} \in (-\infty, 0] \Rightarrow 0 \leq V < 1 : & I_x < I_z \\ \in [0, 1) \Rightarrow V < 0 : & I_x < 0 \text{ nothing real} \\ \lim_{m \downarrow 1} V \rightarrow \infty : & I_x < 0 \text{ nothing real} \\ \in (1, \infty) \Rightarrow V > 1 : & I_x > I_z \end{array} \right.$$

3.4. Berechnung von I_x

Nach Gleichung (3.5) kann bei Kenntnis von I_z und $m := \frac{\omega_F}{\Omega}$ das Trägheitsmoment I_x bestimmt werden. $m = (173 \pm 5)$ wurde aus der Steigung des linearen Fits von $\Omega(\omega_F)$ aus den Messwerten von Tabelle 2.3 bestimmt.

$$I_x = \frac{m I_z}{m - 1} \quad \Delta I_x = I_x \sqrt{\frac{\Delta I_z^2}{I_z^2} + \frac{\Delta m^2}{m^2 (m - 1)^2}} \quad (3.5)$$

$$I_x = (4.98 \pm 0.15) \cdot 10^{-3} \text{ kg m}^2$$

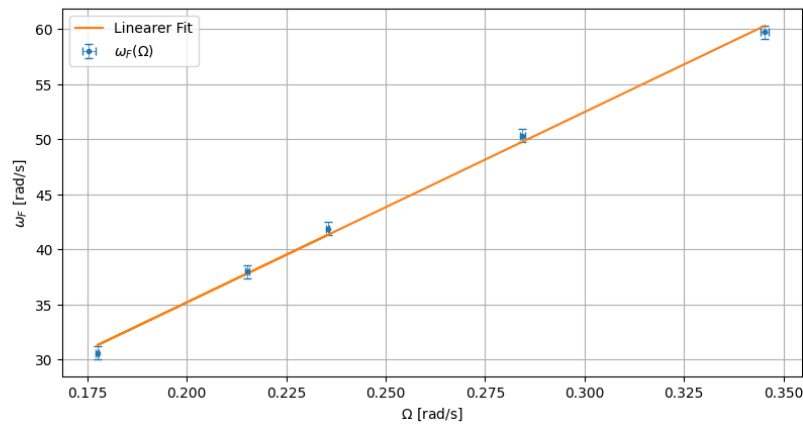


Abb. 3.3: Fitten einer linearen Funktion an $\omega_F(\Omega)$

3.5. BERECHNUNG VON I_X AGAIN

Aus Gleichung (1.2) folgt, dass wenn $m := \frac{\omega_N}{\omega_F}$ die Steigung eines linearen Fits an $\omega_N(\omega_F)$ ist (an die Messwerte aus Tabelle 2.4), so ist:

$$I_x = \frac{I_z}{m} \qquad \Delta I_x = I_x \sqrt{\frac{\Delta I_z^2}{I_z^2} + \frac{\Delta m^2}{m^2}} \quad (3.6)$$

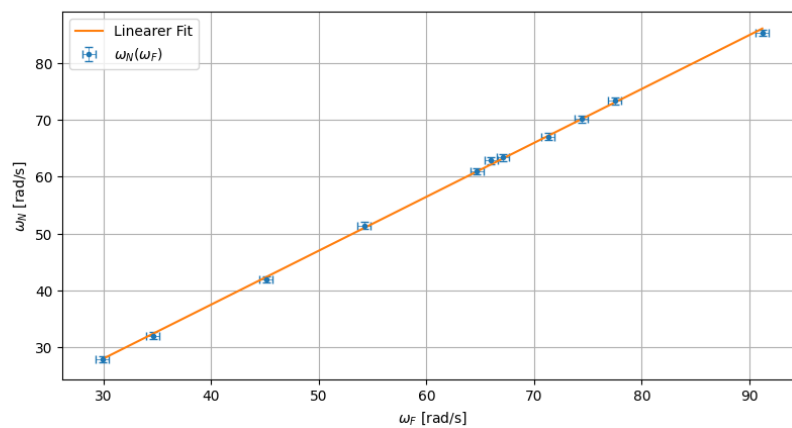


Abb. 3.4: Fitten einer linearen Funktion an $\omega_N(\omega_F)$

$$I_x = (5.21 \pm 0.16) \text{ kg m}^2$$

4. Zusammenfassung und Diskussion

4.1. Fitfehler

Bei jeder einzelnen Aufgabe sollten aus linearen Fits Größen gewonnen werden, die zur Rechnung notwendig waren. NUR ist hier das Problem, dass ein standardmäßiger `curvefit(lin)` einer linearen Funktion `lin` mit der least-squares Methode berechnet wird. Dies beinhaltet nur Fehler in `ydata`, den auf der y-Achse aufgetragenen Messwerten. Die extra berechneten Fehler der `xdata` Sets fließen also überhaupt nicht in (Un-)Genauigkeit der Fitergebnisse ein.

4.2. Dämpfung

Ob die Messwerte dem realen Versuchsaufbau entsprechen, wäre interessant zu wissen. Sinnvoll wäre es unter Umständen gewesen, die Dämpfung auch bei anderen Gewichtsverteilungen zu testen, um zu prüfen, ob die Übertragung des Dämpfungskoeffizienten (der unter Belastung mit zwei Gewichtsstücken ermittelt wurde) auf die anderen Konfigurationen gerechtfertigt ist.

Es gibt in allen Fits keine außergewöhnlichen Messpunkte, die auf einen systematischen Fehler bei einer bestimmten Frequenz hinweisen würden, alle linearen Verläufe sehen gut aus.

4.3. Berechnung von I_x

Im Skript steht eigentlich, dass man den linearen Fit an $\Omega(\omega_F)$ legen soll, jedoch ist der relative Fehler von $\frac{\Delta\omega_F}{\omega_F} > \frac{\Delta\Omega}{\Omega}$. Ein einfacher linearer `curvefit` ignoriert die Unsicherheiten der x-Werte, deswegen habe ich mit der Steigung $m = \frac{\omega_F}{\Omega}$ weitergerechnet. Am Ende habe ich beide Methoden verglichen, und es kommt nach Signifanten Stellen dasselbe raus, OBWOHL der relative Fehler von $\frac{\Delta m_1}{m_1} = 2.9\%$ und $\frac{\Delta m_2}{m_2} = 0.6\%$. Also war die Überlegung schon gut, komischerweise trägt sie beim Endergebnis keine Früchte.

Ah, ich hab herausgefunden, warum. Wenn man Gleichung Gleichung (3.5) betrachtet, und die einzelnen Summanden ausrechnet, so wird sichtbar, dass der hauptauschlaggebende Fehler nicht von m kommt, sondern von I_z .

4.4. Vergleich der Methoden zur I_x Bestimmung

Die Abweichung der berechneten Ergebnisse beläuft sich auf 1.1σ . Dies ist keine nennenswerte Abweichung. Jedoch sollte man bemerken, dass die erste Methode einen Wert ergab, der nur $0.03 \cdot 10^{-3} \text{ kg m}^2$ höher als der $I_z = (4.95 \pm 0.14) \text{ kg m}^2$ Wert liegt, und diese Werte somit innerhalb des anderen Wertes Fehlergrenzen liegen. Die Bedingung $I_x > I_z$ ist also nur knapp erfüllt. Für diese Methode sagt die Steigung $m = 175 \pm 3$ nach $V = \frac{m}{m-1}$ klar aus, dass das Verhältnis V zwischen $I_x = V \cdot I_z$ nur knapp größer 1 ist.

Ein Grund für die Abweichung der zwei Methoden ist ziemlich sicher der Wechsel der Messgeräte: Für die Ermittlung von f_F in Tabelle 2.3 wurde das Laser/Infrarotlicht betriebene Frequenzmessgerät genutzt (PeakTech 2780), für die Ermittlung von ω_F in Tabelle 2.4 jedoch das Stroboskop. Während beim Stroboskop ein Mensch als Sensor fungiert und die entsprechende Frequenz einstellt, wird diese beim Laser automatisch gemessen - die hier vorliegende Nutation führt jedoch dazu, dass der Lichtstrahl nicht immer perfekt zurück auf den Sensor reflektiert wird, was eine fehlerhafte Frequenzmessung zur Folge hat.

4.5. Fazit

Schöner Versuch, Kreiselphänomene wurden gut aufgearbeitet, man kann sich nun besser etwas unter den Effekten vorstellen. Die Theorie und die Vorversuche waren sehr interessant.

Der Meme über Hubble stammt übrigens schon aus PAP1, der ist mir also nicht erst beim Lesen der Versuchsmotivation im Skript eingefallen, in der Hubble explizit erwähnt wird :) Was mich aber interessieren würde, hast du den Meme verstanden?

Abbildungsverzeichnis

0.1. <i>Meme</i> : Hubble's reaction wheels	1
1.1. Versuchsaufbau	4
1.2. Nutation eine skräftefreien und symmetrischen Kreisel	5
1.3. Farbscheibe zur Bestimmung von Ω^2	6
1.4. Präzession eines schweren Kreisels	6
3.1. Aufgabe 1: $\ln(\omega_F(t))$ Fit	10
3.2. Aufgabe 2: Präzession	11
3.3. Aufgabe 4: $\omega_F(\Omega)$ Fit	13
3.4. Aufgabe 5: Nutationsfrequenz	13

Tabellenverzeichnis

2.1. Reibungsmessung	8
2.2. Präzessionsfrequenz	8
2.3. Nutation - Ω Messung	9
2.4. Nutation - Stroboskopmessung	9

Literaturquellen

²Dr. J. Wagner, *Physikalisches Praktikum 2.1 für Studierende der Physik B.Sc.* [Online; Stand 03/2026] (Universität Heidelberg, März. 2026), S. 18–28, https://git.physi.uni-heidelberg.de/schwierz/Versuchsanleitungen/src/branch/main/PAP2_1.pdf (besucht am 11.03.2026) (siehe S. 4–6, 12, 16, 18).

Abbildungsquellen

¹Chamillionaire feat. Krayzie Bone, *Ridin'*, (2006) (siehe S. 1).

³ESA/Hubble, M.Kornmesser/L. Calçada, *Hubble's reaction wheels*, <https://esahubble.org/videos/hubblecast114d/> (besucht am 11.03.2026) (siehe S. 1).

A. Formeln der statistischen Analyse

Für die statistische Auswertung von n Messwerten x_i werden folgende Größen definiert [2]:

$$\text{Arithmetisches Mittel} \quad \bar{x} = \frac{1}{n} \sum_{i=1}^n x_i \quad (\text{A.1})$$

$$\text{Varianz} \quad \sigma^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2 \quad (\text{A.2})$$

$$\text{Standardabweichung} \quad \sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2} \quad (\text{A.3})$$

$$\text{Fehler des Mittelwerts} \quad \Delta \bar{x} = \frac{\sigma}{\sqrt{n-1}} \quad (\text{A.4})$$

$$\text{Gauß'sche Fehlerfortpfl.} \quad \Delta f = \sqrt{\sum_{i=1}^N \left(\frac{\partial f}{\partial x_i} \Delta x_i \right)^2} \quad (\text{A.5})$$

$$\text{relativer Fehler } f = xy / f = x/y \quad \frac{\Delta f}{|f|} = \sqrt{\left(\frac{\Delta x}{x} \right)^2 + \left(\frac{\Delta y}{y} \right)^2} \quad (\text{A.6})$$

$$\text{Absolute Abweichung} \quad A = |a_{lit} - a_{gem}| \quad \Delta A = \sqrt{(\Delta a_{lit})^2 + (\Delta a_{gem})^2} \quad (\text{A.7})$$

$$\text{Relative Abweichung} \quad R = \frac{A}{a_{lit}} \quad (\text{A.8})$$

$$\text{Signifikanz} \quad S = \frac{A}{\Delta A} = \frac{|a_{lit} - a_{gem}|}{\sqrt{\Delta a_{lit}^2 + \Delta a_{gem}^2}} \quad (\text{A.9})$$

B. Code-Anhang

Bei den Funktionen habe ich teilweise Dinge von verschiedenen Altprotokollen zusammengesucht, und auf meine Bedürfnisse umgeschrieben. Die konkreten Berechnungen sind dann eigenständig.

April 3, 2026

```
[1]: # Numpy für bessere Berechnungen
import numpy as np
from numpy import exp, sqrt, log, pi
from uncertainties import unumpy as unp

# Weiteres für bessere Rechnungen
import pylab as py

from decimal import Decimal, ROUND_CEILING, getcontext # Besonders für sig.
↳ Runden
import math

# Berechnungen und Plotting
from scipy import odr
import scipy.optimize
import scipy.constants as c
from scipy.optimize import curve_fit
from scipy.stats import chi2
from scipy.stats import poisson
from scipy.integrate import quad
from scipy.signal import find_peaks
from scipy.signal import argrelextrema, argrelemin, argrelemax
from scipy.special import factorial

%matplotlib widget
import matplotlib.pyplot as plt
import matplotlib.mlab as mlab
import matplotlib.transforms as transforms

# Zum Auslesen von Dateien und ähnlichem
import os
import os.path
from pathlib import Path

import pandas as pd # Auch wichtig für den Latex Export
from pytexit import py2tex
import csv
```

```

import re
from typing import List

# Besseres Funktionen handling
import sympy as sp
from sympy import separatevars

# Display und Output
from IPython.display import display, Math, Latex, HTML

# Grafiken in EU-Größen ausgeben
def figsize_cm(width_cm, height_cm):
    return (2*width_cm / 2.54, 2*height_cm / 2.54)
def comma_to_float(valstr):
    return float(valstr.replace(',', '.'))

```

```

[2]: currentversuch = "213"
base = Path.cwd()
Messwerte_path = Path(base.parent.parent.parent/"Messwerte"/currentversuch)
↳ # initializing paths
Ausgabe_path = Path(base/"Diagramme")
print(Messwerte_path)
print(Ausgabe_path)

```

c:\Users\samue\OneDrive\Dokumente\PAP\PAP2.1\Messwerte\213

c:\Users\samue\OneDrive\Dokumente\PAP\PAP2.1\Auswertungen\Code\213\Diagramme

Runden signifikanter Stellen

```

[3]: def round_to_sigs(val, errVal, einheiten):
    """
        Funktion zur Rundung eines Fehlers und die Anpassung des Messwertes daran.
        ↳ Diese Funktion wurde etwas umständlicher
        geschrieben, da python mit Floats und Runden schnell in Probleme rennt.
        ↳ Daher musste hier mit dezimal gearbeitet werden.
        Zudem sollten besonders kleine und große Messwerte in der
        ↳ Dezimalschreibweise geschrieben werden, damit diese auch
        für Protokolle geeignet sind.

        Parameter
        -----
        **val** : float
            Messwert

        **errVal** : float
            Ungenauigkeit des Messwertes
    """

```

```

Return
-----
**value_rounded** : str
    Gerundeter Messwert

**error_rounded** : str
    Gerundete Ungenauigkeit des Messwertes

**res** : str
    "Messwert \pm Fehler"
"""

# Daten zu Dezimal wechseln, da Floats probleme machen
val = Decimal(str(val))
errVal = Decimal(str(errVal))

exp = int(math.floor(math.log10(float(errVal)))) # Die erste
↳signifikante Stellenposition wird anhand des errVal bestimmt

if round(float(errVal / (Decimal(10) ** exp))) < 3: # wenn 1,2,3
↳erste signifikante Stelle, dann kommt eine zweite Nachkommastellenposition
↳hinzu
    exp -= 1

scale = Decimal(10) ** exp

val_round = (val / scale).quantize(Decimal('1'), rounding=ROUND_CEILING) *
↳scale # mit scale wird das Komma so verschoben, dass alle
↳signifikanten Stellen vor dem Komma stehen, und dann alle Nachkommastellen
↳weggerundet werden
err_round = (errVal / scale).quantize(Decimal('1'), rounding=ROUND_CEILING)
↳* scale

res = f"\\qty{{{val_round}}({err_round})}{{{einheiten}}}"
return float(val_round), float(err_round), res

# wissenschaftliche Notation erstmal vernachlässigen (da häufig Einheiten
↳gewollt sind, die 10~/e Prefixe beinhalten)

```

Gauss'sche Fehlerfortpflanzung

```

[4]: def gff(function, errPronePar, latex=True):
    """
    Kann die Fehlerformel einer gegebenen Gleichung bestimmen.

    Parameters
    -----

```

```

**func** : sympy function
    Funktion dessen Fehler bestimmt werden soll.

**errPronePar** : Array
    Liste (Array) aller fehlerbehafteten Größen der Gleichung con sp.Symbols
    Diese Werte werden als x_sym, y_sym, z_sym etc. bezeichnet und sind
    ↪ungleich den Werten für x, y, z.
    Für die Werte wird daher die Bezeichnung x_val, y_val, z_val etc.
    ↪genutzt und für deren Fehler err_x, err_y, err_z etc.

Return
-----
**absolut_err** : sympy function
    Gibt die Fehlergleichung des absoluten Fehlers wieder.

**relativ_err** : sympy function
    Gibt die Fehlergleichung des relativen Fehlers wieder.

**errProneParamters** : array
    Liste aller Fehlerbehafteten Größen
    ""

error = 0
errProneParamaters = []

function = sp.sympify(function)
variables = errPronePar.split(",")
variables = sp.symbols(variables)

for variable in variables:
    Delta = sp.Symbol(f"Delta_{variable}")
    partial = sp.diff(function, variable)           # Die Funktion
    ↪wird nach der fehlerbehafteten Variable abgeleitet
    term=sp.UnevaluatedExpr(partial*Delta)**2
    error = error + ((term))                         # Fehler werden
    ↪quadratisch aufsummiert
    errProneParamaters.append((variable,Delta))

absolute_err=sp.simplify(sp.sqrt(error),rational = True)
relative_err=sp.simplify(sp.sqrt(error/function**2),rational = True)

if latex:
    #Prints out the Latex Code for the Functions
    print("Funktion:")
    display(Math(sp.latex(function,long_frac_ratio=2)))
    print(sp.latex(function))
    print("Absoluter Fehler:")

```

```

display(Math(sp.latex(absolute_err,long_frac_ratio=2)))
print(sp.latex(absolute_err))
print("Relativer Fehler:")
display(Math(sp.latex(relative_err,long_frac_ratio=2)))
print(sp.latex(relative_err))
return function,absolute_err, relative_err, errProneParamaters

```

Sigma-Abweichungen

```

[5]: def sigma_abweichung(p1, p2, err_p1 = 0, err_p2 = 0):
    """
        Funktion zum berechnen der Sigma-Abweichung von zwei Messwerten, oder einem
        ↪Messwert und einem Literaturwert.
    """
    if err_p1 == 0 and err_p2 == 0:
        raise ValueError("Für die Sigma-Abweichung muss mindestens ein Wert
        ↪fehlerbehaftet sein!")
    else:
        abweichung=Decimal(str(abs(p1 - p2)/(np.sqrt(err_p1 ** 2+err_p2 ** 2))))

        exp = int(math.floor(math.log10(float(abweichung))))
        if round(float(abweichung / (Decimal(10) ** exp))) < 3:           # wenn
        ↪1,2,3 erste signifikante Stelle, dann kommt eine zweite
        ↪Nachkommastellenposition hinzu
            exp -= 1
            scale = Decimal(10) ** exp
            abweichung_round = (abweichung / scale).quantize(Decimal('1'),
            ↪rounding=ROUND_CEILING) * scale
            res = f"\num{{{abweichung_round}}}"

        return float(abweichung_round),res

```

AUFGABE 1: REIBUNG

```

[20]: zeiten=np.array([0,60,120,180,240,300,360])
delta_zeiten=2
w_F=np.array([650,596,543,499,458,422,386])*2*c.pi/60
delta_w_F=5*2*c.pi/60

v_round = np.vectorize(round_to_sigs, otypes=[float, float, object],
    ↪excluded=['einheiten'])
w_F,delta_w_F,_ = v_round(w_F.astype(float), delta_w_F,einheiten=r"\per\s")

plt.close('all')
fig, ax = plt.subplots(figsize=figsize_cm(12,6))
ax.set_title('Reibungsverlust')
ax.set_ylabel(r'$\omega_F$ [rad/s]')
ax.set_xlabel(r'Zeit $t$ [s]')

```

```

ax.set_yscale('log')
ax.set_ylim((3e1,1*1e2))
# ax.set_xlim(())
ax.errorbar(x=zeiten,y=w_F,xerr=delta_zeiten,yerr=delta_w_F,fmt="
↵",capsize=3,elinewidth=0.5,label='w_F(t) unter Reibungsverlusten.')

def lin(x,a,b):
    return a*x + b
def exp(x,a,b):
    return b*np.exp(a*x)

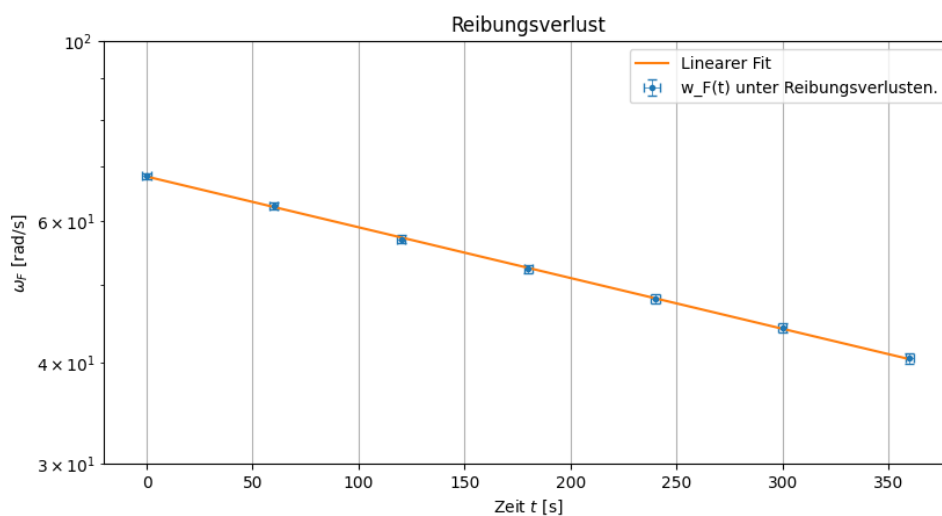
popt1, pcov1 = curve_fit(lin, zeiten, np.log(w_F), sigma = delta_w_F/
↵w_F,absolute_sigma=True) # Fehler wird durch Logarithmus verändert, aus ggf.
↵folgt log_fehler=fehler/messwerte
ax.plot(zeiten, exp(zeiten, popt1[0],np.exp(popt1[1])),r'-', label='Linearer
↵Fit')
damp, Delta_damp =round_to_sigs(-popt1[0], np.
↵sqrt(pcov1[0][0]),einheiten=r"\per\s")[0],round_to_sigs(-popt1[0], np.
↵sqrt(pcov1[0][0]),einheiten=r"\per\s")[1]
print(damp,Delta_damp)
T12, Delta_T12 = (np.log(2)/damp,np.log(2)/damp**2 * Delta_damp)
print(round_to_sigs(T12,Delta_T12,r"\s")[0],round_to_sigs(T12,Delta_T12,r"\s")[1])

ax.grid()
ax.legend()
plt.show()
plt.savefig(Ausgabe_path/"Nr.1_Reibung.png",format="png")

```

0.00145 4e-05

479.0 14.0



AUFGABE 3.1: UMRECHNUNG DER DATEN

```
[7]: T_P=np.array([75.8,107.2,49.9,142.9,99.7,56.6,50.2,36.6,23.3,64.4,49.2,27.9])
Delta_T_P=0.5
w_P=2*c.pi/T_P
Delta_w_P=2*c.pi*Delta_T_P/T_P**2
w_P,Delta_w_P,_ = v_round(w_P.astype(float), Delta_w_P,einheiten=r"\per\s")

w_F_i=np.array([493,694,313,682,490,269,670,490,315,630,495,280])*2*c.pi/60
Delta_w_F_i=5*2*c.pi/60
w_F_i,Delta_w_F_i,_=v_round(w_F_i.astype(float),Delta_w_F_i,einheiten=r"\per\s")
gff("w_F_i*e**(-damp*T_P)","w_F_i,damp,T_P")

print(f"{w_P=}")
print(f"{Delta_w_P=}")
print(f"{w_F_i=}")
print(f"{Delta_w_F_i=}")
w_F_f=w_F_i/np.exp(T_P*damp)
Delta_w_F_f= sqrt((Delta_T_P**2*damp**2*w_F_i**2 +
    ↳Delta_damp**2*T_P**2*w_F_i**2 + Delta_w_F_i**2)/np.exp(2*T_P*damp))
w_F_f,Delta_w_F_f,_ = v_round(w_F_f.astype(float),↳
    ↳Delta_w_F_f,einheiten=r"\per\s")
print(f"{w_F_f=}")
print(f"{Delta_w_F_f=}")
bar_w=(w_F_i+w_F_f)/2
Delta_bar_w=1/2*np.sqrt(Delta_w_F_i**2+Delta_w_F_f**2)
bar_w,Delta_bar_w,_ = v_round(bar_w.astype(float),↳
    ↳Delta_bar_w,einheiten=r"\per\s")
print(f"{bar_w=}")
print(f"{Delta_bar_w=}")
```

Funktion:

$$e^{-T_P \text{damp}} w_{Fi}$$

$$e^{\{-T_P \text{damp}\} w_{Fi}}$$

Absoluter Fehler:

$$\sqrt{e^{-2T_P \text{damp}} \left(\Delta_{TP}^2 \text{damp}^2 w_{Fi}^2 \log(e)^2 + \Delta_{\text{damp}}^2 T_P^2 w_{Fi}^2 \log(e)^2 + \Delta_{wFi}^2 \right)}$$

$$\sqrt{e^{\{-2T_P \text{damp}\}} \left(\Delta_{TP}^2 \text{damp}^2 w_{Fi}^2 \log(e)^2 + \Delta_{\text{damp}}^2 T_P^2 w_{Fi}^2 \log(e)^2 + \Delta_{wFi}^2 \right)}$$

Relativer Fehler:

$$\sqrt{\Delta_{TP}^2 \text{damp}^2 \log(e)^2 + \Delta_{\text{damp}}^2 T_P^2 \log(e)^2 + \frac{\Delta_{wFi}^2}{w_{Fi}^2}}$$

```

\sqrt{\Delta_{T_P}^2 \text{damp}^2 \log{\left(e \right)}^2 + \Delta_{\text{damp}}^2 T_P^2 \log{\left(e \right)}^2 + \frac{\Delta_{w_F}^2}{w_F^2}}
w_P=array([0.0829 , 0.0587 , 0.126 , 0.04397, 0.0631 , 0.1111 , 0.1252 ,
           0.1717 , 0.27 , 0.0976 , 0.1278 , 0.226 ])
Delta_w_P=array([0.0006 , 0.0003 , 0.0013 , 0.00016, 0.0004 , 0.001 , 0.0013 ,
                 0.0024 , 0.006 , 0.0008 , 0.0013 , 0.005 ])
w_F_i=array([51.7, 72.7, 32.8, 71.5, 51.4, 28.2, 70.2, 51.4, 33. , 66. , 51.9,
             29.4])
Delta_w_F_i=array([0.6, 0.6, 0.6, 0.6, 0.6, 0.6, 0.6, 0.6, 0.6, 0.6, 0.6, 0.6])
w_F_f=array([46.4, 62.3, 30.6, 58.2, 44.5, 26. , 65.3, 48.8, 32. , 60.2, 48.4,
             28.3])
Delta_w_F_f=array([0.6, 0.6, 0.6, 0.6, 0.6, 0.6, 0.6, 0.6, 0.6, 0.6, 0.6, 0.6])
bar_w=array([49.1, 67.5, 31.7, 64.9, 48. , 27.1, 67.8, 50.1, 32.5, 63.1, 50.2,
             28.9])
Delta_bar_w=array([0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5])

```

```

[8]: plt.close('all')
fig, ax = plt.subplots(figsize=figsize_cm(12,6))
ax.set_title(' ')
ax.set_ylabel(r'Präzessionszeit $T_P$ [s]')
ax.set_xlabel(r'Frequenz $\bar{\omega}_F$ [rad/s]')
# ax.set_ylim((3e1,1*1e2))
# ax.set_xlim(())
massen=[r'$m-20$ cm',r'$m-15$ cm',r'$2m-20$ cm',r'$2m-15$ cm']
n=3
j=0
a=[]
Delta_a=[]
for i in range(0, len(T_P), n):
    ax.errorbar(x=bar_w[i : i + n],y=T_P[i : i + n],xerr=Delta_bar_w[i : i + n],yerr=Delta_T_P,fmt=".",capsize=3,elinewidth=0.5,label=f"$T_P$ {massen[j]}")
    popt, pcov = curve_fit(lin, bar_w[i : i + n], T_P[i : i + n], sigma =Delta_T_P,absolute_sigma=True)
    ax.plot(bar_w[i : i + n],lin(bar_w[i : i + n], popt[0],popt[1]),r'--',color=f"C{j}",label=f"Linearer Fit {massen[j]}")
    print(f"$a_{massen[j]}$={round_to_sigs(popt[0],np.sqrt(pcov[0][0])),einheiten=r"$s^2$"}")
    a.append(round_to_sigs(popt[0],np.sqrt(pcov[0][0])),einheiten=r"$s^2$")
    Delta_a.append(round_to_sigs(popt[0],np.sqrt(pcov[0][0])),einheiten=r"$s^2$")
    j+=1

ax.grid()
ax.legend()
plt.show()
plt.savefig(Ausgabe_path/"Nr.Präzession.png",format="png")

```

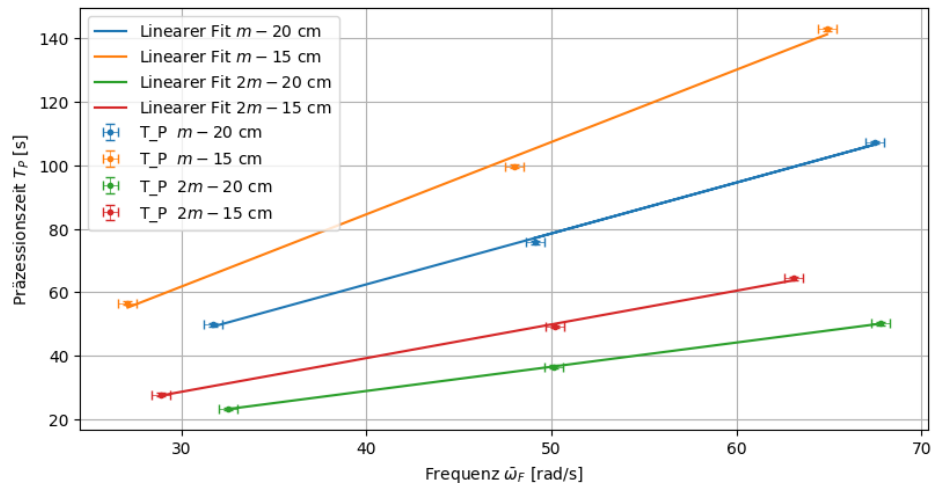
```

a_$m-20$ cm=\qty{1.602(0.020)}{\s\squared}
a_$m-15$ cm=\qty{2.275(0.019)}{\s\squared}

```

$$a_{2m-20} \text{ cm} = \sqrt{0.763(0.021)} \text{ s}^2$$

$$a_{2m-15} \text{ cm} = \sqrt{1.061(0.021)} \text{ s}^2$$



AUFGABE 3.2: I_Z BERECHNUNG

```
[9]: a=np.array(a)
Delta_a=np.array(Delta_a)
g=9.80984
Delta_g=0.00002
l=np.array([0.2,0.15,0.2,0.15])
Delta_l=0.003
m=np.array([9.85,9.85,2*9.85,2*9.85])*1e-3
I_z=a*g*l*m/(2*c.pi)
Delta_I_z=I_z*np.sqrt(Delta_a**2/a**2 + Delta_g**2/g**2 + Delta_l**2/l**2)
I_z,Delta_I_z,latex_I_z= v_round(I_z.astype(float), Delta_I_z,r"\kg\m\squared")
for i, line in enumerate(latex_I_z):
    print(f"I_{{{massen[i]}}}= {line}") # Fügt am Ende das LaTeX-Zeilenende \\
    ↪ hinzu
bar_I_z=np.sum(I_z)/4
stdIz = np.std(I_z)
Delta_barIz = np.sqrt((np.sum(Delta_I_z)/4)**2+(stdIz/4)**2)
print(round_to_sigs(bar_I_z,Delta_barIz,r"\kg\m\squared")[2])
```

$$I_{m-20} \text{ cm} = \sqrt{0.00493(0.00010)} \text{ kg}\cdot\text{m}^2$$

$$I_{m-15} \text{ cm} = \sqrt{0.00525(0.00012)} \text{ kg}\cdot\text{m}^2$$

$$I_{2m-20} \text{ cm} = \sqrt{0.00470(0.00015)} \text{ kg}\cdot\text{m}^2$$

$$I_{2m-15} \text{ cm} = \sqrt{0.00490(0.00014)} \text{ kg}\cdot\text{m}^2$$

$$\sqrt{0.00495(0.00014)} \text{ kg}\cdot\text{m}^2$$

```
[10]: gff("a*m*g*l/(2*pi)", "a,g,l")
```

Funktion:

$$\frac{aglm}{2\pi}$$

$\frac{a g l m}{2 \pi}$

Absoluter Fehler:

$$\frac{1}{2\pi} \sqrt{m^2 (\Delta_a^2 g^2 l^2 + \Delta_g^2 a^2 l^2 + \Delta_l^2 a^2 g^2)}$$

$\frac{\sqrt{m^2 (\Delta_a^2 g^2 l^2 + \Delta_g^2 a^2 l^2 + \Delta_l^2 a^2 g^2)}}{2 \pi}$

Relativer Fehler:

$$\sqrt{\frac{\Delta_a^2}{a^2} + \frac{\Delta_g^2}{g^2} + \frac{\Delta_l^2}{l^2}}$$

$\sqrt{\frac{\Delta_a^2}{a^2} + \frac{\Delta_g^2}{g^2} + \frac{\Delta_l^2}{l^2}}$

```
[10]: (a*g*l*m/(2*pi),
      sqrt(m**2*(Delta_a**2*g**2*l**2 + Delta_g**2*a**2*l**2 +
      Delta_l**2*a**2*g**2))/(2*pi),
      sqrt(Delta_a**2/a**2 + Delta_g**2/g**2 + Delta_l**2/l**2),
      [(a, Delta_a), (g, Delta_g), (l, Delta_l)])
```

AUFGABE 5.1: BERECHUNG m_l für I_x

```
[11]: T_w=np.array([18.2,22.1,29.2,35.4,26.7])
w=2*pi/T_w
Delta_T_w=0.5/10
Delta_w=2*c.pi*Delta_T_w/T_w**2
w,Delta_w,_ = v_round(w.astype(float), Delta_w,r"\per\s")
w_F=np.array([570,480,362,292,400])*2*c.pi/60
Delta_w_F=5*2*c.pi/60
w_F,Delta_w_F,_=v_round(w_F.astype(float),Delta_w_F,r"\per\s")

plt.close('all')
fig, ax = plt.subplots(figsize=figsize_cm(12,6))
ax.set_xlabel(r'\Omega$ [rad/s]')
ax.set_ylabel(r'\omega_F$ [rad/s]')
# ax.set_ylim((3e1,1*1e2))
# ax.set_xlim(())
ax.errorbar(x=w,y=w_F,xerr=Delta_w,yerr=Delta_w_F,fmt=".",
            ↵",capsize=3,elinewidth=0.5,label=r'\omega_F(\Omega)$')

popt, pcov = curve_fit(lin, w,w_F, sigma =Delta_w_F,absolute_sigma=True)
ax.plot(w, lin(w, popt[0], popt[1]),r'-', label='Linearer Fit')
```

```

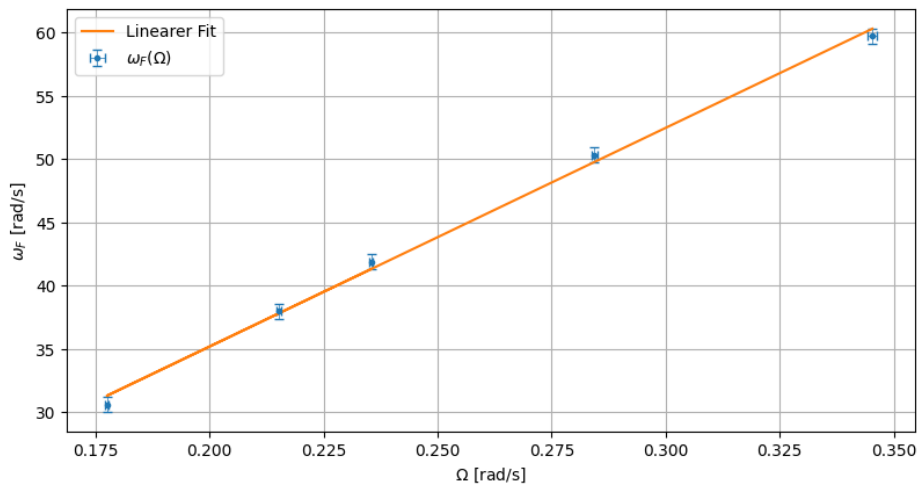
m=round_to_sigs(popt[0],np.sqrt(pcov[0][0]),r"[0]
Delta_m=round_to_sigs(popt[0],np.sqrt(pcov[0][0]),r"[1]

print(round_to_sigs(popt[0],np.sqrt(pcov[0][0]),r"[2])
ax.grid()
ax.legend()
plt.show()
plt.savefig(Ausgabe_path/"Nr.4_w-w_F.png",format="png")

print(max(Delta_w/w))
print(max(Delta_w_F/w_F))

```

\qty{173(5)}{}



0.0028960324355632784

0.0196078431372549

AUFGABE 5.1: BERECHNUNG I_x

```

[12]: bar_I_z,Delta_barIz=round_to_sigs(bar_I_z,Delta_barIz,r"\kg\m\squared")[0],round_to_sigs(bar_I
I_x=-bar_I_z*m/(1 - m)
Delta_I_x =I_x*sqrt(Delta_barIz**2/bar_I_z**2 + Delta_m**2/(m**2*(m - 1)**2))
print(round_to_sigs(I_x,Delta_I_x,r"\kg\m\squared")[2])
print(Delta_m**2/(m**2*(m - 1)))
gff("-m*I_z/(1-m)","m,I_z")

```

\qty{0.00498(0.00015)}{\kg\m\squared}

4.856454850122033e-06

Funktion:

$$-\frac{I_z m}{1-m}$$

$$-\frac{I_z}{m} \frac{m}{1-m}$$

Absoluter Fehler:

$$\sqrt{\frac{1}{(m-1)^4} (\Delta_{I_z}^2 m^2 (m-1)^2 + \Delta_m^2 I_z^2)}$$

$$\sqrt{\frac{\Delta_{I_z}^2 m^2 (m-1)^2 + \Delta_m^2 I_z^2}{(m-1)^4}}$$

Relativer Fehler:

$$\sqrt{\frac{\Delta_{I_z}^2}{I_z^2} + \frac{\Delta_m^2}{m^2 (m-1)^2}}$$

$$\sqrt{\frac{\Delta_{I_z}^2}{I_z^2} + \frac{\Delta_m^2}{m^2 (m-1)^2}}$$

```
[12]: (-I_z*m/(1 - m),
      sqrt((Delta_I_z**2*m**2*(m - 1)**2 + Delta_m**2*I_z**2)/(m - 1)**4),
      sqrt(Delta_I_z**2/I_z**2 + Delta_m**2/(m**2*(m - 1)**2)),
      [(m, Delta_m), (I_z, Delta_I_z)])
```

AUFGABE 5 AGAIN: BERECHNUNG m2 und I_x2

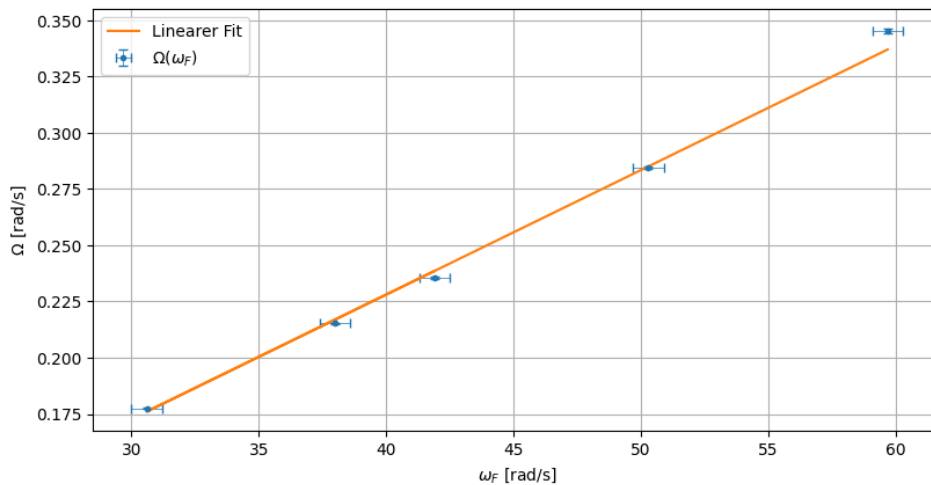
```
[13]: plt.close('all')
fig, ax = plt.subplots(figsize=figsize_cm(12,6))
ax.set_ylabel(r'$\Omega$ [rad/s]')
ax.set_xlabel(r'$\omega_F$ [rad/s]')
# ax.set_ylim((3e1, 1*1e2))
# ax.set_xlim(())
ax.errorbar(x=w_F, y=w, xerr=Delta_w_F, yerr=Delta_w, fmt=".",
            capsize=3, elinewidth=0.5, label=r'$\Omega(\omega_F)$')

popt, pcov = curve_fit(lin, w_F, w, sigma =Delta_w, absolute_sigma=True)
ax.plot(w_F, lin(w_F, popt[0], popt[1]), r'-', label='Linearer Fit')
m2=round_to_sigs(popt[0], np.sqrt(pcov[0][0]), r"") [0]
Delta_m2=round_to_sigs(popt[0], np.sqrt(pcov[0][0]), r"") [1]
print(round_to_sigs(popt[0], np.sqrt(pcov[0][0]), r"") [2])
ax.grid()
ax.legend()

barz, Delta_barIz=round_to_sigs(bar_I_z, Delta_barIz, r"\kg\m\squared") [0], round_to_sigs(bar_I_z,
I_x2=barz/(1-m2)
Delta_I_x2 = sqrt((Delta_barIz**2*(m2 - 1)**2 + Delta_m2**2*barz**2)/(m2 -
1)**4)
print(round_to_sigs(I_x2, Delta_I_x2, r"\kg\m\squared") [2])
print(sigma_abweichung(I_x, I_x2, Delta_I_x, Delta_I_x2))
```

```
plt.show()
```

```
\qty{0.00553(0.00003)}{}
\qty{0.00498(0.00015)}{\kg\m\squared}
(0.007, '\num{0.007}')
```



```
[14]: gff("I_z/(1-m)", "I_z,m")
```

Funktion:

$$\frac{I_z}{1-m}$$

$\frac{I_z}{1-m}$

Absoluter Fehler:

$$\sqrt{\frac{1}{(m-1)^4} (\Delta_{I_z}^2 (m-1)^2 + \Delta_m^2 I_z^2)}$$

$$\sqrt{\frac{\Delta_{I_z}^2}{I_z^2} + \frac{\Delta_m^2}{(m-1)^2}}$$

Relativer Fehler:

$$\sqrt{\frac{\Delta_{I_z}^2}{I_z^2} + \frac{\Delta_m^2}{(m-1)^2}}$$

$$\sqrt{\frac{\Delta_{I_z}^2}{I_z^2} + \frac{\Delta_m^2}{(m-1)^2}}$$

```
[14]: (I_z/(1 - m),
      sqrt((Delta_I_z**2*(m - 1)**2 + Delta_m**2*I_z**2)/(m - 1)**4),
```

```
sqrt(Delta_I_z**2/I_z**2 + Delta_m**2/(m - 1)**2),
[(I_z, Delta_I_z), (m, Delta_m)])
```

AUFGABE 6: BERECHUNG I_x AUF ZWEITEN WEG

```
[15]: w_F=np.array([870,740,710,680,640,630,617,517,430,330,285])*2*c.pi/60
Delta_w_F=Delta_w_N=2*c.pi*5/60
w_N=np.array([815,700,670,640,605,600,582,490,400,305,265])*2*c.pi/60
w_F,Delta_w_F,_ = v_round(w_F.astype(float), Delta_w_F,r"\per\s")
w_N,Delta_w_N,_=v_round(w_N.astype(float),Delta_w_N,r"\per\s")

plt.close('all')
fig, ax = plt.subplots(figsize=figsize_cm(12,6))
ax.set_ylabel(r'$\omega_N$ [rad/s]')
ax.set_xlabel(r'$\omega_F$ [rad/s]')
# ax.set_ylim((3e1,1*1e2))
# ax.set_xlim(())
ax.errorbar(x=w_F,y=w_N,xerr=Delta_w_F,yerr=Delta_w_N,fmt=".",
            ↵, capsize=3,elinewidth=0.5,label=r'$\omega_N(\omega_F)$')

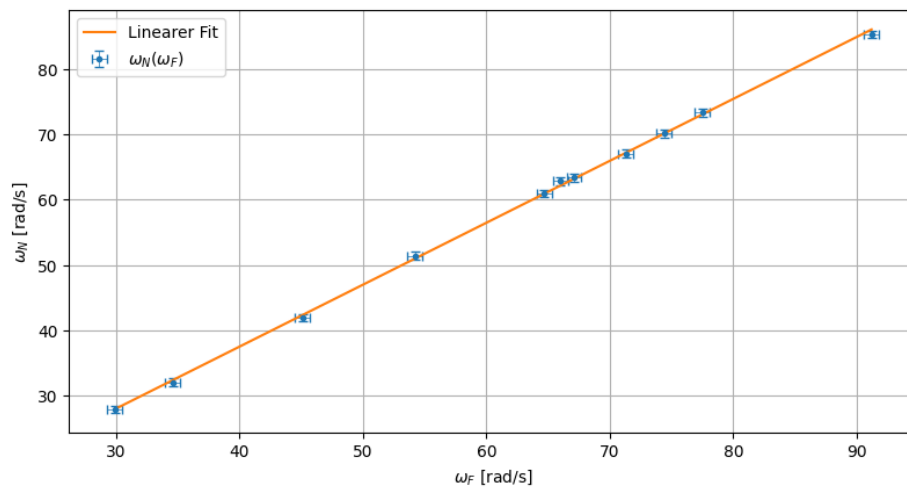
popt, pcov = curve_fit(lin, w_F,w_N, sigma =Delta_w_N,absolute_sigma=True)
ax.plot(w_F, lin(w_F, popt[0], popt[1]),r'-', label='Linearer Fit')
m=round_to_sigs(popt[0],np.sqrt(pcov[0][0]),r"")[0]
Delta_m=round_to_sigs(popt[0],np.sqrt(pcov[0][0]),r"")[1]
print(m,Delta_m)
print(bar_I_z,Delta_barIz)
ax.grid()
ax.legend()
I_x=bar_I_z/m
Delta_I_x=I_x* sqrt(Delta_barIz**2/bar_I_z**2 + Delta_m**2/m**2)
print(round_to_sigs(I_x,Delta_I_x,r"\kg\m\squared")[2])

plt.show()
plt.savefig(Ausgabe_path/"Nr.5_w_N-w_F.png",format="png")
```

0.951 0.011

0.00495 0.00014

\qty{0.00521(0.00016)}{\kg\m\squared}



```
[16]: gff("I_z/m", "I_z, m")
```

Funktion:

$$\frac{I_z}{m}$$

$\frac{I_z}{m}$

Absoluter Fehler:

$$\sqrt{\frac{1}{m^4} (\Delta_{I_z}^2 m^2 + \Delta_m^2 I_z^2)}$$

$$\sqrt{\frac{\Delta_{I_z}^2}{I_z^2} + \frac{\Delta_m^2}{m^2}}$$

Relativer Fehler:

$$\sqrt{\frac{\Delta_{I_z}^2}{I_z^2} + \frac{\Delta_m^2}{m^2}}$$

$$\sqrt{\frac{\Delta_{I_z}^2}{I_z^2} + \frac{\Delta_m^2}{m^2}}$$

```
[16]: (I_z/m,
      sqrt((Delta_I_z**2*m**2 + Delta_m**2*I_z**2)/m**4),
      sqrt(Delta_I_z**2/I_z**2 + Delta_m**2/m**2),
      [(I_z, Delta_I_z), (m, Delta_m)])
```

```
[21]: sigma_abweichung(4.98,5.21,0.16,0.15)
```

```
[21]: (1.1, '\\num{1.1}')
```